

Synthèse Prometheus

Mise en place d'une chaîne d'observabilité complète sur un cluster Kubernetes local, pour le projet **DevOps Portfolio MERN**. De l'instrumentation du code applicatif jusqu'à la notification par email.

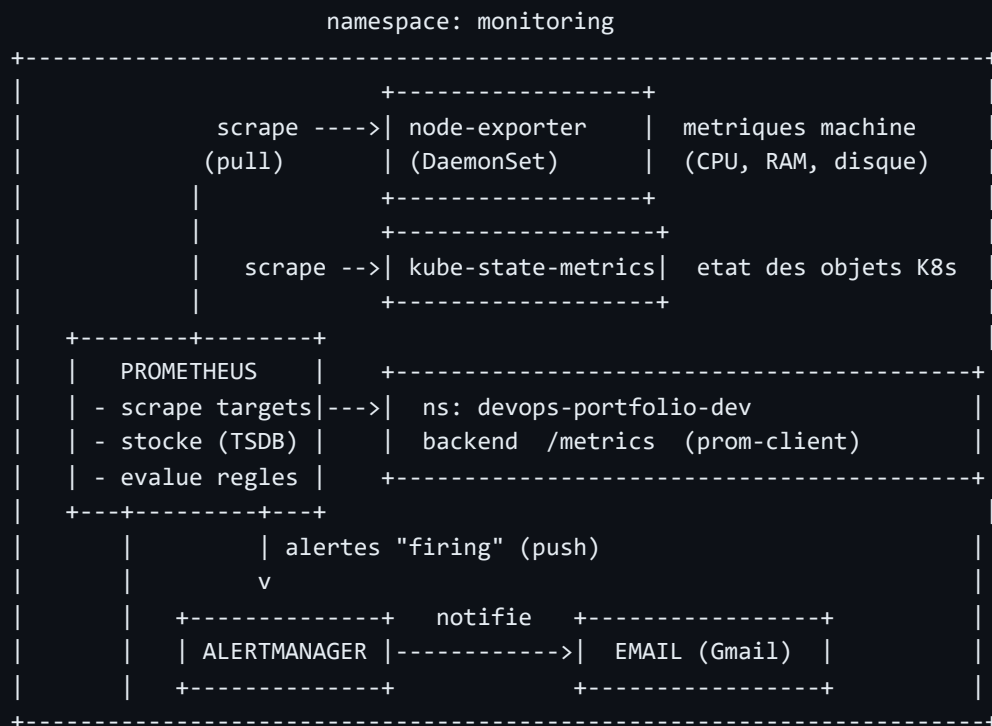
Niveau Apprentissage / portfolio Cluster Kubernetes local (kind) Namespace **monitoring**

1. Objectif & vue d'ensemble

Prometheus est un système de **surveillance (monitoring)** basé sur les **métriques**. Son principe fondateur : il **tire** (modèle *pull*) les mesures en interrogeant périodiquement ses cibles, les stocke dans une base de séries temporelles, permet de les interroger (PromQL), d'évaluer des règles d'alerte, et délègue l'envoi des notifications à Alertmanager.

Analogie : Prometheus est un infirmier qui fait le tour des chambres toutes les 15 secondes, relève les constantes de chaque patient (CPU, RAM, requêtes...) et les note dans un cahier horodaté.

2. Architecture mise en place



3. Concepts clés

Concept	Définition
Modèle pull	Prometheus va <i>chercher</i> les métriques en appelant les cibles. Il ne reçoit rien de façon passive.
Target	Une cible à scraper : une adresse <code>host:port</code> exposant un endpoint de métriques.
Scrape	L'action de relever les métriques d'une target (ici toutes les 15 s via <code>scrape_interval</code>).
Endpoint <code>/metrics</code>	La « porte » HTTP par laquelle une cible expose ses mesures au format texte Prometheus.
Série temporelle	Le modèle de données : <code>nom_metrique{labels} valeur</code> enregistré avec un horodatage.
Labels	Étiquettes clé=valeur qui identifient/dimensionnent une série (ex. <code>job</code> , <code>instance</code> , <code>route</code>). Permettent de filtrer et d'agréger.
Exporter	Un programme qui traduit l'état d'un système en métriques Prometheus (ex. node-exporter pour la machine).
Règle d'alerte	Une expression PromQL + une condition + une durée <code>for</code> . Évaluée en continu.
Alertmanager	Composant séparé qui reçoit les alertes <i>firing</i> et gère routing, regroupement, déduplication, silences et notifications.

Les 4 types de métriques

Type	Comportement	Exemple
Counter	Ne fait qu'augmenter (remis à 0 au redémarrage)	<code>http_requests_total</code>
Gauge	Monte et descend (valeur instantanée)	<code>node_memory_MemAvailable_bytes</code>
Histogram	Répartition par paliers (buckets)	<code>http_request_duration_seconds</code>
Summary	Quantiles calculés côté client	(non utilisé ici)

Cycle de vie d'une alerte

inactive (condition fausse) → **pending** (condition vraie, mais pas encore depuis la durée `for`) → **firing** (condition vraie assez longtemps → envoi à Alertmanager).

4. Composants déployés

Composant	Rôle	Fichiers
Prometheus	Scrape, stockage TSDB, évaluation des règles	<code>monitoring/prometheus/01..05</code>
node-exporter	Métriques de la machine (DaemonSet)	<code>monitoring/node-exporter/</code>
kube-state-metrics	État des objets Kubernetes (+ RBAC)	<code>monitoring/kube-state-metrics/</code>
backend instrumenté	Expose <code>/metrics</code> via <code>prom-client</code> (RED)	<code>backend/src/middleware/metrics.js</code>
Alertmanager	Routing + notifications email (SMTP Gmail)	<code>monitoring/alertmanager/</code>

Le mot de passe SMTP n'est **jamais** dans Git : il est stocké dans un **Secret Kubernetes** (`alertmanager-smtp`) monté en fichier, et lu via `smtp_auth_password_file` .

5. PromQL — recettes utiles

```
# La cible répond-elle ? (1 = oui)
up

# RAM utilisée en %
(1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100

# Nombre de pods Running, par namespace
sum by (namespace) (kube_pod_status_phase{phase="Running"})

# Taux de requêtes par seconde sur le backend (counter -> rate)
rate(http_requests_total[5m])

# Latence moyenne par route
rate(http_request_duration_seconds_sum[5m])
/ rate(http_request_duration_seconds_count[5m])

# Part d'erreurs 5xx
sum(rate(http_requests_total{status_code=~"5.."}[5m]))
/ sum(rate(http_requests_total[5m]))
```

6. Commandes utiles

```
# Appliquer / mettre à jour un composant
kubectl apply -f monitoring/prometheus/

# Recharger Prometheus après un changement de ConfigMap
kubectl rollout restart deployment/prometheus -n monitoring

# Accéder aux UIs (cluster kind => port-forward, pas NodePort)
kubectl port-forward -n monitoring svc/prometheus 9090:9090
kubectl port-forward -n monitoring svc/alertmanager 9093:9093

# Déclencher l'alerte TargetDown pour tester (puis remettre à 1)
kubectl scale deployment/backend-deployment -n devops-portfolio-dev --replicas=0
kubectl scale deployment/backend-deployment -n devops-portfolio-dev --replicas=1

# Vérifier les Alertmanagers connus de Prometheus (API)
kubectl exec -n monitoring deploy/prometheus -- wget -qO- http://localhost:9090/api/v1/alertmanagers
```

7. Pièges rencontrés & bonnes pratiques

Piège	Leçon
<code>npm ci</code> échoue après ajout d'une dépendance	Toujours committer <code>package-lock.json</code> à jour en même temps que <code>package.json</code> .
Quality Gate SonarQube bloque le pipeline	Le nouveau code doit être couvert par des tests. Garde-fou utile, on ajoute un test plutôt que baisser le seuil.
NodePort inaccessible	Sur un cluster <code>kind</code> , les NodePorts ne sont pas exposés sur <code>localhost</code> sans config : utiliser <code>port-forward</code> .
Cardinalité des labels	Utiliser le <i>motif</i> de route (<code>/:id</code>) plutôt que l'URL exacte, sinon explosion du nombre de séries.
Stockage <code>emptyDir</code>	Les données Prometheus sont perdues si le pod est recréé. Pour de la persistance : un PVC (étape ultérieure).
Secret SMTP	Ne jamais mettre un mot de passe dans un ConfigMap/Git. Secret K8s + montage fichier.
Saturation du cluster mono-node	Ajouter toute la stack monitoring sature un node unique (pods <code>Pending</code> faute de CPU, Mongo tué par OOM). Réduire les <code>requests</code> CPU et utiliser <code>strategy: Recreate</code> (évite d'héberger 2 pods en même temps lors d'un redéploiement).
Stateful + PVC ReadWriteOnce	Pour une base (Mongo) avec volume RWO, <code>RollingUpdate</code> provoque un deadlock (le nouveau pod ne peut pas monter le volume tenu par l'ancien). <code>strategy: Recreate</code> est indispensable.
Une métrique « vide » a révélé un vrai bug	L'absence de <code>auth_login_attempts_total</code> a permis de découvrir que l'app ne pouvait pas lire MongoDB (URI sans identifiants alors que l'auth était activée). C'est le but de l'observabilité : rendre visible l'invisible.

8. Glossaire express

Terme	En une phrase
TSDB	Time-Series DataBase : la base où Prometheus stocke les séries.
Scrape interval	Fréquence à laquelle Prometheus relève les cibles.
RED	Rate, Errors, Duration : les 3 métriques clés d'un service web.
DaemonSet	Objet K8s garantissant 1 pod par node.
RBAC	Droits d'accès K8s (ServiceAccount + ClusterRole + binding).
Silence	Mise en sourdine temporaire d'une alerte dans Alertmanager.

9. Prochaines étapes

- **Grafana** : data source Prometheus + dashboards visuels (étape suivante).
- **Persistence** : ajouter un PVC à Prometheus.
- **Service discovery** : remplacer les targets statiques par la découverte automatique des pods K8s.
- **Industrialisation** : déployer la stack via Terraform (provider Helm) pour rester cohérent avec l'IaC du projet.